PAPER ONE OF THREE

Dynamic Recursive Tree RAG

A Self-Evolving Hierarchical Memory Architecture for Agentic AI

Alex d'Anjou

April 2026

Preprint — Not Peer Reviewed

ABSTRACT

Current agentic AI systems face a fundamental bottleneck: longitudinal reasoning. Vector-based retrieval excels at local fact-finding but collapses when confronted with global questions requiring thematic understanding across months or years of interaction history. Static hierarchical approaches such as RAPTOR improve upon flat retrieval but suffer from passive vector spray across abstraction layers, losing both precision and navigational intent.

This paper introduces Dynamic Recursive Tree RAG (DRT-RAG), a self-evolving memory architecture that addresses these failures through three coordinated innovations: (1) a multi-channel node schema that separates facts from narrative to resist information degradation; (2) a dynamically growing recursive tree (formally a DAG; see Section 3.4) with automatic layer generation that is designed to ensure the global memory map fits within a configured working-context budget as history grows; and (3) an agentic Librarian controller — the LLM itself — that navigates the tree top-down with explicit intent classification rather than relying on passive vector similarity.

This paper is presented as a systems architecture proposal — a position paper in the tradition of database systems research where novel architecture designs with strong formal properties are evaluated on their structural guarantees rather than empirical benchmarks alone. We position DRT-RAG against representative retrieval, graph, and agentic-memory systems (including recent LongMemEval-optimized architectures), identify its critical limitations with mitigations, and propose an evaluation framework. We further argue that DRT-RAG is most powerfully understood not as a complete memory solution but as the episodic memory tier in a larger three-tier memory architecture — the subject of subsequent papers in this series.

1. Introduction

The promise of agentic AI — systems that act as long-term companions, advisors, and collaborators — depends critically on memory. Not the shallow retrieval of recent context, but genuine longitudinal understanding: the ability to know that a user's priorities shifted in March, that a decision made in October contradicts a commitment made in January, that a pattern emerging over two years is more significant than any individual data point within it.

Today's systems cannot do this reliably. The dominant approach, Retrieval-Augmented Generation (RAG), treats an agent's history as a pile of undifferentiated text chunks. When asked a global question — 'How has my relationship with risk changed since I started this company?' — the system grabs the five chunks with the highest cosine similarity to the query. These chunks are almost certainly not the ones that answer the question. They are statistically similar strings, not thematically coherent context.

The AI research community has recognized this problem. RAPTOR introduced recursive summarization into a static tree. MemGPT proposed hierarchical memory tiers with explicit management. GraphRAG added community detection and summary hierarchies to graph structures. Each is a genuine improvement. None fully solves the combination of *unbounded history with bounded working context (under configured budgets)*, top-down navigational intent, and resistance to summary degradation that long-term agentic memory requires.

DRT-RAG synthesizes and extends these approaches. The central insight is that the LLM itself — not a vector search engine — should be the navigator of its own memory. Given a sufficiently accurate global map, an LLM can reason about which branches of its history are relevant to a query, drill down with targeted precision, and stop when it has what it needs. The tree makes the map. The LLM reads the map. The result is memory that reasons rather than memory that retrieves.

A useful way to understand the current landscape is as two partially separable research tracks. **Track 1** optimizes episodic recall: how accurately can a system retrieve facts from long conversation histories at inference time. LongMemEval is the best standardized measure of this track. **Track 2** targets identity formation and behavioral continuity: can a system develop stable behavioral dispositions that persist without retrieval, resist adversarial pressure, and evolve coherently over time. LongMemEval does not measure this track. DRT-RAG is primarily a Track 1 architecture, but it is designed explicitly to be *consolidation-ready* — an episodic substrate with provenance and governance semantics that downstream Track 2 mechanisms can safely learn from.

1.1 The Three Failure Modes We Target

Semantic Fragmentation: Flat RAG returns isolated facts without the thematic context that gives them meaning. A query about career satisfaction returns entries mentioning satisfaction, not the longitudinal arc that shows how it changed.

Context Pollution: Multi-layer vector search across all abstraction levels simultaneously returns a mix of raw logs and high-level summaries, polluting the context window with redundant and conflicting granularity.

Static Hierarchy Blindness: RAPTOR-style trees built once and searched passively cannot adapt to new information, handle contradictions, or route with intent. The tree knows what it contained when it was built, not what has changed since.

2. Related Work and Positioning

DRT-RAG is not a wholesale replacement for prior work. It is a synthesis and extension, and positioning it honestly against the existing landscape is important for evaluating its claims.

2.1 April 2026 Landscape (LongMemEval-Optimized Systems)

Since LongMemEval (Wu et al., ICLR 2025), a fast-moving ecosystem of memory systems has emerged whose primary objective is benchmark-grade episodic recall (Track 1). Representative systems with strong *reported* LongMemEval-S results include agentmemory V4 (McCann, 2026; GitHub report; single-run result reported as 96.20% = 481/500 on LongMemEval-S under the author's stated "real-retrieval" configuration with oracle/direct-context disabled), Mastra Observational Memory (OM) (Barnes, 2026; vendor report; 84.23% with GPT-4o and 94.87% with the model identified in the post as "GPT-5-mini", under a stable, cacheable context window), Hindsight (Latimer et al., 2025; preprint; reported up to 91.4% with an architecture organized into four epistemically distinct networks), Zep/Graphiti (Rasmussen et al., 2025; preprint; reported 71.2% with temporal knowledge graph retrieval), and TiMem (Li et al., 2026; preprint; reported 76.88% on LongMemEval-S using a temporal-hierarchical tree). These figures should be treated as directional rather than a strict leaderboard: across publications they vary with backbone model, dataset subset (S vs. M), evaluation harness and judge configuration, and whether direct-context/oracle-access conditions are excluded (and some reports use slightly different naming conventions, e.g., "LongMemEval_S"). The conservative conclusion is that, under retrieval-optimized configurations, multiple systems now report performance in the high-80s to mid-90s on the current LongMemEval-S task, suggesting the benchmark is approaching a practical ceiling and shifting differentiation toward robustness and substrate semantics.

DRT-RAG is not a leaderboard entry in this paper; it is a substrate proposal with explicit structural guarantees (provenance, invariants, deletion semantics) that many benchmark-optimized systems do not treat as first-class. For this reason, we position DRT-RAG against two comparison sets: (i) historical lineage systems that established key ideas (e.g., hierarchical summarization and agentic retrieval), and (ii) contemporary LongMemEval-optimized systems that represent the current engineering frontier for episodic recall. The core claim of DRT-RAG is not maximal accuracy, but that a memory system can be engineered to preserve an auditable epistemic chain from verbatim experience to summarized belief while still remaining retrieval-competitive.

Pragmatically, DRT-RAG also incorporates several retrieval lessons that have emerged from the 2026 frontier. In particular, we treat **hybrid lexical+dense retrieval** and **lightweight re-ranking** as baseline engineering for precision recall; we allow **threshold-triggered**

compression in addition to session boundaries; and we encode explicit temporal metadata in the Ledger (observed_at / event_at / relative_time) to reduce time-resolution errors. What distinguishes DRT-RAG is not that it is the only system to use these techniques, but that it embeds them inside a memory substrate with explicit *governance semantics*: span-anchored provenance, non-upgrading certainty, a current-state arbitration layer (the Ledger), provenance-aware deletion propagation, and an explicit design goal of serving as a consolidation-ready episodic substrate.

2.2 Flat RAG

Standard retrieval-augmented generation encodes document chunks as dense vectors and retrieves the top-k chunks by cosine similarity at query time. It is computationally cheap, well-understood, and excellent at local fact retrieval. Its failure mode is global reasoning: it has no structure above the chunk level, so questions requiring synthesis across time or theme are answered by statistical coincidence rather than architectural design.

2.3 RAPTOR

Recursive Abstractive Processing for Tree-Organized Retrieval builds a summary tree by recursively clustering and summarizing leaf-level chunks into parent nodes. It substantially improves global reasoning by creating abstraction layers. Its limitation is that the tree is built once and searched passively — the retrieval system runs vector similarity across all layers simultaneously, which reintroduces context pollution. The tree provides structure; it does not provide navigation.

2.4 Self-RAG

Self-RAG (Asai et al., ICLR 2024 Oral) introduced adaptive retrieval as a learned behavior rather than a fixed pipeline step. The model generates special reflection tokens to decide whether to retrieve, what to retrieve, and how to critique retrieved content before generating a final response. Self-RAG demonstrated that LLM-driven retrieval decisions outperform static retrieve-then-generate pipelines on knowledge-intensive tasks. Its limitation for longitudinal memory is that it operates on document corpora rather than structured interaction histories, has no hierarchical summarization, and its self-reflection tokens are session-scoped rather than persistent. It establishes the principle that LLM participation in retrieval improves outcomes; DRT-RAG extends that principle to a memory architecture with long-term structure.

2.5 Agentic RAG Survey (Singh et al., 2025)

The 2025 Agentic RAG survey (Singh et al., 2025) established a formal taxonomy of agentic retrieval architectures: single-agent, multi-agent, hierarchical, and graph-based. It identifies the key differentiating dimension as whether the LLM participates in retrieval decisions or is a passive recipient of retrieved content. DRT-RAG falls into the hierarchical agentic category by this taxonomy, and the survey's finding that agentic retrieval consistently outperforms passive retrieval on complex multi-hop queries directly supports DRT-RAG's core architectural choice. We note the survey does not address the trust invariants, provenance semantics, or deletion

architecture that DRT-RAG introduces — it covers retrieval strategy, not memory substrate integrity.

2.6 A-RAG (Du et al., 2026)

A-RAG (Du et al., 2026) is the most architecturally proximate prior work to DRT-RAG and requires explicit positioning. A-RAG exposes hierarchical retrieval as a tool interface — keyword search, semantic search, and chunk read — that an agent can call selectively based on query intent. Its empirical evaluation demonstrates that agents with hierarchical retrieval tool access substantially outperform fixed-pipeline RAG on multi-hop and complex reasoning tasks.

DRT-RAG and A-RAG share the core insight that LLM-controlled hierarchical retrieval outperforms passive retrieval. The architectural distinction is in what is being retrieved from. A-RAG operates on static document corpora, exposing their structure as a retrieval interface. DRT-RAG operates on a continuously growing, agent-managed memory substrate — the dynamic DAG — that does not exist at deployment time but is built and maintained by the system over the course of its operation. The retrieval tools in A-RAG navigate a fixed document index. The Librarian in DRT-RAG navigates a self-organizing memory that it has helped to construct, with trust invariants, a Current State Ledger, provenance-aware deletion semantics, and a multi-channel node schema that A-RAG does not address. DRT-RAG's contribution is the memory substrate architecture, not the agentic retrieval loop, which A-RAG has established as effective.

2.7 MemGPT

MemGPT introduces tiered memory management — main context, external storage, and retrieval — with the LLM acting as its own memory manager through explicit function calls. It demonstrates that LLM-controlled memory management is feasible and effective. It does not, however, implement hierarchical summarization or dynamic layer generation, leaving it without support for *unbounded history with bounded working context (under configured budgets)* as longitudinal agentic memory demands.

2.8 GraphRAG

Microsoft's GraphRAG builds a knowledge graph over input documents, performs community detection to identify thematically related clusters, and generates hierarchical community summaries. Its global query mode substantially outperforms flat RAG on narrative corpora. Its limitations for agentic memory are that it is designed for static document corpora rather than continuously growing interaction histories, and its community detection is a batch process rather than an incremental one.

2.9 A-MEM (NeurIPS 2025)

A-MEM (Xu et al., 2025; NeurIPS 2025) proposes an agentic memory system using Zettelkasten-style interconnected note networks, where the agent dynamically organizes memories by creating, linking, and updating notes based on new experiences. A-MEM is the closest prior work to DRT-RAG in the agentic memory rather than agentic retrieval space — it

explicitly addresses the problem of memory that grows and reorganizes with agent experience rather than indexing a static corpus. Its limitation is that it does not implement hierarchical progressive summarization, dynamic layer generation, a trust invariant architecture, a Current State Ledger, or provenance-aware deletion semantics. DRT-RAG and A-MEM converge on the same problem space from different starting points; A-MEM from knowledge graph organization, DRT-RAG from hierarchical summarization with integrity guarantees.

2.10 Generative Agents (2023)

Park et al. (2023) introduced Generative Agents — interactive simulacra of human behavior that maintain a memory stream of observations, reflections, and plans, with a retrieval function combining recency, importance, and relevance. Generative Agents is the foundational prior work for agentic persistent memory and introduced several concepts this series builds on: the memory stream as a continuous accumulation substrate, reflection as a distinct processing stage that synthesizes higher-order insights from raw observations, and retrieval as a function of multiple signals rather than vector similarity alone. Its limitation relative to DRT-RAG is that the memory stream is flat — it does not implement recursive hierarchical summarization, dynamic layer generation, or the trust invariants and provenance architecture that DRT-RAG treats as core structural properties. The observation-reflection-plan cycle in Generative Agents is the conceptual ancestor of DRT-RAG's L0-to-LN progressive summarization; the key architectural difference is that DRT-RAG makes this hierarchy dynamic and formally structured rather than a flat time-ordered stream.

Several of DRT-RAG’s design choices are also visible as convergent solutions in contemporary benchmark-driven systems. Mastra OM arrives at progressive compression (Observer → Reflector) as a practical necessity for long-horizon performance, supporting the general thesis that memory architectures must explicitly manage what survives summarization. Hindsight separates memory into multiple epistemic networks (e.g., facts vs. experiences vs. beliefs), echoing the motivation of DRT-RAG’s multi-channel node schema: if evidence and inference are mixed, they will become indistinguishable after repeated compression. These convergences do not imply architectural equivalence. DRT-RAG’s differentiators are not primarily retrieval accuracy, but provenance, invariants, and ledger semantics: it is designed to make memory *governable* (auditable, deletable, and safe to consolidate) rather than merely retrievable.

2.11 DRT-RAG's Differentiator

The comparison below is intentionally split across two tables. Table 2.11 compares DRT-RAG to historical lineage and architecturally proximate systems on broad structural properties such as hierarchy, navigation, and dynamic growth. Table 2.12 then compares DRT-RAG to contemporary LongMemEval-optimized systems on consolidation-readiness and governability semantics — including provenance, certainty discipline, current-state arbitration, and deletion propagation — which are central to this paper’s contribution.

System	Hierarchical	Dynamic Growth	LLM Navigation	Multi-Channel Nodes	Trust Invariants	Continuous Update
Flat RAG	No	N/A	No	No	No	Append-only
RAPTOR	Yes	No	No	No	No	Batch rebuild
Self-RAG	No	No	Yes — adaptive	No	No	Session-scoped
MemGPT	Partial	No	Yes	No	No	Yes
GraphRAG	Yes	No	No	No	No	Batch rebuild
A-RAG	Yes	No	Yes — tool loop	No	No	Static corpus
A-MEM	No	Yes — note links	Yes	No	No	Yes
DRT-RAG	Yes	Yes — automatic	Yes — map-first	Yes	Yes	Yes — async

2.12 Consolidation Readiness (Substrate Semantics)

System	Span-anchored provenance to L0	Certainty non-upgrading invariant	Current State Ledger + conflict set	Provenance-aware deletion propagation	Designed as consolidation substrate
agentmemory V4	Partial	Not stated	Not stated	Not stated	No
Mastra OM	Partial	Not stated	Partial	Not stated	No
Hindsight	Partial (epistemic separation; not span IDs)	Not stated	Not stated	Not stated	No
Zep/Graphiti	Partial	Not stated	Partial	Partial	No
TiMem	Not stated	Not stated	Not stated	Not stated	No
DRT-RAG	Yes	Yes	Yes	Yes	Yes

Interpretation note: this table is intentionally conservative. “Not stated” means we did not find the feature claimed as a first-class architectural property in the system’s public paper, repository, or documentation at the time of writing (it may still be implementable). “Partial” indicates a related idea is present, but not in a form equivalent to DRT-RAG’s invariants (e.g., epistemic separation into memory types is not the same as span-anchored provenance to verbatim L0, and a reflection/update mechanism is not the same as an explicit non-upgrading

certainty invariant). DRT-RAG's "Yes" ratings are grounded in explicit requirements in this paper: span anchoring (Section 3.1), non-upgrading certainty and provenance invariants (Section 3.3), the Ledger fast-path with conflict sets (Section 3.2), provenance-aware deletion propagation (Section 5.1), and the explicit role of DRT-RAG as the episodic substrate for consolidation in the broader series (Sections 1 and 7).

3. DRT-RAG Architecture

3.1 Multi-Channel Node Schema

The most critical architectural decision in DRT-RAG is the node schema. Prior hierarchical systems store summaries as prose. Prose summaries degrade through recursive summarization — the 'telephone game' problem where nuance, contradiction, and precision are progressively lost across abstraction layers.

DRT-RAG addresses this by decomposing every node at every layer into five distinct channels. Each channel has different compression properties and different retrieval utility:

Channel	Contents	Compression Behavior	Primary Use
Fact Table	Named entities, dates, decisions, commitments, numbers	Resistant — structured data survives summarization	Precision recall, contradiction detection
State Claims	Time-bounded assertions: 'As of [date], user X was Y'	Resistant if time-scoped	Current state reasoning, update tracking
Open Loops	Unresolved items, pending decisions, tracked goals	Moderate — monitored for resolution	Continuity, follow-through
Evidence Pointers	UUIDs of child nodes + snapshotted supporting verbatim quotes/spans	Preserved exactly — never summarized	Audit trail, attribution, regression testing
Narrative Summary	Human-readable prose summary of the period or topic	Degrades predictably — expected	Global orientation, routing decisions

The key insight is that the Narrative Summary is allowed to degrade — that's its job. It gives the LLM a readable map to navigate with. But factual precision is preserved in the Fact Table and anchored by Evidence Pointers back to verbatim Layer 0 source material. The telephone game cannot corrupt what has never been summarized.

In this sense, DRT-RAG implements *reflective layering* as a persistent memory artifact: higher-layer Narrative Summaries and State Claims function as the system's reflection step

(synthesizing meaning and salience from raw experience), while the Fact Table and Evidence Pointers ensure that reflection remains auditably grounded in verbatim L0 evidence. This is intentionally analogous to Observer→Reflector pipelines (e.g., Mastra OM) and the observation→reflection stage in Generative Agents, but expressed as an explicit, navigable hierarchy rather than as a flat stream.

Critical implementation invariant: Fact Table entries must be derived from quoted spans or span identifiers from L0 source material — not inferred from context or generated from narrative summaries. Any Fact Table item without a supporting verbatim span must be tagged as 'inference' rather than 'fact.' This prevents hallucinated structured data from carrying false authority upward through the hierarchy. An inferred entity that gets treated as a fact by Layer 3 is more dangerous than an uncertain narrative summary precisely because structured data is trusted by default. The span-anchoring requirement eliminates that failure mode at the source.

3.2 The Current State Ledger

A significant limitation of pure tree architectures is the update cascade problem. If a high-level summary contains the assertion 'User is happily married to Sarah' and a new interaction introduces a divorce, correcting the tree requires rewriting the daily summary, which invalidates the monthly summary, which invalidates the yearly summary. This is computationally expensive and architecturally fragile.

DRT-RAG addresses this with a separate Current State Ledger — a lightweight, flat key-value store maintained alongside the tree. The Ledger tracks the most recent known value of any identified entity relationship, preference, or ongoing state. It is updated incrementally with each new interaction, independently of the tree structure.

The router receives the Ledger as part of its base context alongside the top-layer tree summary. Current-state queries are answered primarily from the Ledger; historical queries navigate the tree. State contradictions flag the relevant tree nodes as 'dirty' for lazy re-evaluation during the next maintenance cycle, rather than triggering immediate cascade rewrites.

The Ledger is small by design — it contains only current state, not history. Its entries are structured assertions with mandatory fields: a *status* (ACTIVE or PENDING_RESOLUTION), an *observation timestamp* (when the system observed the evidence), an *event timestamp* (when the event is claimed to have occurred, if stated), an optional *relative-time field* preserving unresolved temporal references (e.g., “last week”), a provenance pointer to the DRT-RAG node(s) that support the assertion, a confidence level (HIGH/MEDIUM/LOW based on evidence quality), and a conflict set recording what the entry replaced (if anything) and why. A minimal Ledger entry looks like: 'relationship_status: divorced | status: ACTIVE | observed_at: 2026-02-24 | event_at: 2026-02-24 | relative_time: null | provenance: node-uuid-L1-2026-02-24 | confidence: HIGH | replaces: {married, partner: Sarah, since: 2019} | reason: explicit user statement in session'. Critically, LOW-confidence contradictions do not automatically overwrite an existing ACTIVE entry. Instead, the new information is queued as PENDING_RESOLUTION with its own provenance and conflict context, and the agent can arbitrate explicitly at the next appropriate user interaction (e.g., by asking a clarifying question). The change history remains in the tree. The current truth lives in the Ledger. A Ledger entry without provenance is a claim

without a source — and sourceless claims in an authoritative fast-path store become authoritative lies. The provenance requirement is non-negotiable.

3.3 Memory Trust Invariants

A coherent memory system requires explicit trust invariants — rules that govern what each layer is allowed to claim, how claims are verified, and which layers can override others. Without these, the system will silently accumulate trust mismatches: a Narrative Summary treated as ground truth, an inferred Fact Table entry treated as verified, a Ledger entry without provenance treated as authoritative. These failures are silent, which makes them worse than loud failures.

DRT-RAG operates under the following invariants, which are architectural constraints rather than implementation preferences:

- L0 is ground truth. Raw interaction logs are never modified, summarized in-place, or deleted except under explicit user data-deletion requests (see Section 5 on privacy). All other layers derive their authority from L0.
- Evidence Pointers are append-only. Once written, an Evidence Pointer entry cannot be edited in place to point to different evidence. It may be invalidated (flagged with a reason) if the L0 source it references is deleted, and it may be supplemented by additional Evidence Pointer entries if new supporting spans are discovered. Mutable evidence pointers defeat the audit trail.
- Fact Table entries require span anchoring. Every Fact Table item must link to a quoted span or span ID from L0. Items without supporting spans are tagged 'inference' and receive lower routing weight. Inference tags are preserved through summarization — they do not become facts at higher layers.
- Ledger entries require provenance, confidence, and a conflict set. A Ledger entry without all three fields is malformed and must not be used as a routing input.
- Narrative Summaries are non-authoritative orientation tools. They may be used to navigate the tree and orient the Librarian. They must not be used as truth sources without supporting evidence from the Fact Table or Evidence Pointers.
- No layer may increase the certainty of a claim. A claim tagged 'inference' at L1 cannot become 'fact' at L3 simply by surviving summarization. Certainty tags propagate upward; they do not upgrade.

3.4 The Dynamic DAG Structure

DRT-RAG's data structure is formally a directed acyclic graph (DAG) of multi-channel nodes, grounded at Layer 0 in raw interaction logs. We use the term 'tree' informally for readability, but the distinction matters architecturally. A pure tree enforces a single parent per node, which is semantically wrong: a conversation about career direction in March and another in November belong to the same thematic branch and should share a parent summary node, independent of their chronological separation. Forcing them into separate branches purely by calendar creates exactly the boundary fragmentation problem described in Section 4.5.

The DAG parent selection rule is as follows: every node has exactly one primary parent, assigned by temporal proximity for L1 nodes (the node belongs to the period it was created in). Secondary edges — cross-references to thematically related nodes in other branches — are permitted and tracked as non-primary DAG edges. These cross-reference edges do not affect the summarization hierarchy. To prevent secondary-edge density from polluting the map and breaking the working-context budget, secondary edges are treated as *on-demand navigation aids*: they are not injected into the always-present map context by default, and traversal occurs via explicit fetch of cross-references for a given node when needed. During asynchronous maintenance, secondary edges are also subject to pruning/decay (e.g., retaining only the top-k most useful cross-references per node under a utility score), ensuring they remain sparse and informative rather than becoming a dense thematic mesh. Cycles are explicitly prohibited; all edges are directed from child to parent. Provenance tracing follows primary parent edges only, ensuring unambiguous audit trails from any summary node back to its L0 source material.

Secondary edges must propagate provenance and certainty metadata identically to primary-edge navigation: they are a navigation aid, not an authority upgrade. In particular, following a secondary edge may broaden what is inspected, but it must not bypass span-anchoring requirements, must not upgrade certainty tags, and must not introduce Ledger updates without the same provenance constraints.

Layer 0 (L0) — Raw Data: Verbatim interaction transcripts, session logs, or document chunks. Never summarized, never deleted (or archived to cold storage only). The ground truth substrate.

Layer 1 (L1) — Local Summaries: Multi-channel summaries of L0 chunks, generated asynchronously during low-usage periods. Typically daily or per-session boundaries, with semantic boundary detection to avoid splitting coherent topic threads across chunks.

Layer N (Ln) — Progressive Summaries: Each layer summarizes the layer below. The dynamic trigger monitors token count at the topmost layer. When the concatenation of all top-layer nodes exceeds a defined threshold (default: 2,000 tokens), the system proposes Ln+1 creation by clustering top-layer nodes. Clustering is performed by k-means or agglomerative clustering over the Fact Table embeddings of top-layer nodes — using the structured channels rather than narrative prose ensures that clustering reflects conceptual similarity rather than surface-level vocabulary overlap, which is the primary cause of topic drift in layer formation. However, capacity alone is not a sufficient trigger: before generating any Ln+1 parent nodes, the system evaluates the *semantic cohesion* of proposed clusters (e.g., via within-cluster similarity statistics over Fact Table embeddings). If cohesion falls below a configured threshold, the system should avoid forcing an artificial parent narrative and instead either (i) partition the top layer into multiple independent top-level domains (multiple roots) or (ii) defer Ln+1 creation temporarily (allowing the top-layer map to exceed the target token budget within a bounded grace margin) until sufficient cohesion emerges. This threshold is a tuneable parameter; 2,000 tokens was chosen as a conservative default that leaves substantial headroom for the Librarian's reasoning context while ensuring layer generation triggers before the global map becomes unwieldy — it is not a constraint imposed by the architecture but a safe starting point for deployment. This design is intended to ensure the global map remains within a configured

context budget (e.g., reserving ~2k tokens for the map), and assumes the summarizer enforces a strict per-node summary budget at each layer.

Every node at every layer contains full Evidence Pointers to its children, enabling complete traceability from any high-level summary back to the verbatim L0 source. This traceability is a first-class architectural property, not an afterthought — it enables regression testing, auditability, and principled correction when a downstream summary or state claim is challenged.

3.5 The Agentic Librarian: Top-Down Navigation

The defining innovation of DRT-RAG is replacing passive vector similarity with active LLM navigation. The LLM is given the complete top-layer summary as part of its base context — not as retrieved content, but as always-present orientation. The LLM always knows, at a high level, the shape of its history. Detailed retrieval is a deliberate navigation decision, not a statistical accident.

It is important to separate *navigation* from *authority*. The Librarian's role is to decide where to look — which branches to inspect, which tools to invoke, and when to stop — not to upgrade what is true. All truth-bearing updates (especially Ledger writes and Fact Table assertions) remain constrained by the provenance requirements and non-upgrading trust invariants in Section 3.3.

3.5.1 Routing Loop

The routing loop operates as follows:

1. Intent Classification: The LLM classifies the query. Is this a global thematic question ('How has my confidence changed?'), a mid-level synthesis question ('What were the themes of my career conversations last year?'), or a precision needle-in-haystack query ('What was the exact date I decided to leave my job?'). Intent class determines navigation depth.
2. Global Scan with Dual Branch Nomination: The LLM reads the injected top-layer summary and identifies candidate branches via map reasoning. Simultaneously, a lightweight embedding search runs over top-layer node Fact Tables and embeddings to propose candidate branches independently. The Librarian reasons over the union of map-nominated and embedding-nominated branches. This dual nomination is not optional — the map-only approach has a known failure mode where the Librarian confidently excludes branches that the top-layer narrative summary omitted but that contain relevant L0 data. The embedding pass catches these omissions before they become false negatives.
3. Tool Execution — Drill Down: The LLM calls fetch_child_nodes(node_id) to retrieve Ln-1 summaries for all nominated branches. It evaluates whether the answer is available at this abstraction level.
4. Branch-Scoped Hybrid Search: Within selected branches, hybrid search (lexical keyword/BM25-style plus dense embeddings) is run over L0/L1 nodes to catch entities or exact details that may have been omitted from higher-level narrative summaries. This runs in parallel with step 3, not as a fallback.
5. Re-ranking / Commitment Check: Candidate context packets (nodes, spans, and ledger entries) are scored by a lightweight relevance+recency critic. Low-confidence candidates

are rejected unless corroborated by Evidence Pointers or repeated support across channels. This reduces the probability that the Librarian “commits” to a plausible but wrong branch based on a degraded narrative map.

6. Precision Retrieval: If exact quotes, attribution, or verbatim detail are needed, the LLM calls fetch_raw_logs(node_id) using evidence pointers to reach L0. This step is invoked only when the intent classification indicated precision need.
7. Hail Mary Fallback: If tree navigation produces no result, a global L0 vector search runs as a final fallback to catch anything the summary hierarchy omitted.

3.5.2 Anti-Sycophancy Guardrails at the Episodic Tier

Long-horizon memory systems are vulnerable to sycophancy not only in response generation, but in memory selection and elevation: approval-laden spans (e.g., flattering affirmations or over-agreeable interpretations) can be mistakenly treated as stable preferences or authoritative state. DRT-RAG mitigates this at the episodic tier through structural constraints rather than behavioral fine-tuning. First, no layer may increase certainty: candidate preference or state claims extracted from narrative content remain tagged as inference unless anchored to explicit L0 spans and corroborated (e.g., recurrence across time, explicit user-marked importance, or repeated support across channels). Second, the re-ranking and commitment check deprioritizes high-approval but low-evidence packets unless they carry supporting Evidence Pointers. This does not “solve” sycophancy as an alignment problem; it prevents sycophancy from becoming durable memory substrate corruption, and it provides the provenance hooks required for the deeper consolidation-gating mechanisms introduced in Papers Two and Three.

3.5.3 Librarian Retrieval Mechanism — Implementation Notes

The core DRT-RAG architecture is intentionally agnostic to specific retrieval implementations: the trust invariants, provenance model, and ledger semantics do not depend on a particular search algorithm. However, recent LongMemEval-optimized systems suggest several pragmatic retrieval techniques that can be adopted *without* changing DRT-RAG’s epistemic commitments.

For reproducibility, retrieval tool calls (lexical search, embedding search, node fetches, and Evidence Pointer traversal) should be deterministic given the same query and memory state. The model-stochastic components are intent classification and branch nomination; these can be stabilized in practice by using a small routing model at low temperature (and by logging router decisions as part of the audit log described in Section 5.3).

- **Multi-signal hybrid retrieval:** At each node-evaluation step, the Librarian’s tool set should include lexical retrieval (keyword/BM25-style) alongside dense semantic retrieval. This directly improves precision recall for names, dates, IDs, and verbatim phrasing, where pure dense retrieval is brittle.
- **Critic-based re-ranking:** After candidate branches and spans are retrieved, a lightweight critic pass re-ranks candidates for relevance and recency, and enforces a minimum confidence threshold before context injection. Architecturally, this is a small addition; empirically, it can meaningfully reduce false-commitment errors.
- **Temporal three-date handling in the Ledger:** The Ledger’s observed_at / event_at / relative_time split is not just bookkeeping; it is a retrieval feature. Temporal reasoning

failures often arise because “when we learned X” is conflated with “when X happened.” Encoding both fields makes time filtering and update resolution materially easier.

- **Threshold-triggered compression:** In addition to scheduled maintenance, L0→L1 compression can be triggered when uncompressed L0 exceeds a token and/or salience threshold. This avoids waiting for session boundaries and supports stable, cache-friendly prefixes in deployments that prefer in-context memory blocks.

This hybrid retrieval policy — map-first, then branch-scoped hybrid search, then precision retrieval, then global fallback — substantially improves both recall and precision versus either pure agentic navigation or pure vector retrieval alone.

3.6 Routing Cost Optimization

Agentic navigation introduces latency through sequential LLM calls. DRT-RAG addresses this with a tiered model architecture for the routing loop. The intent classifier and drill-down decisions are handled by a small, fast model (equivalent to Claude Haiku or GPT-4o-mini class). Only the final answer generation — after all relevant context has been assembled — uses the full-capability model. This reduces both latency and token cost for the majority of routing operations while preserving answer quality.

3.7 Asynchronous System Maintenance

To prevent user-facing latency from tree maintenance operations, all summarization and tree growth operations are strictly asynchronous. L0 data accumulates during active sessions. L1 summarization runs on a scheduled cron job during low-usage periods. New layer generation is triggered automatically by the token threshold monitor and runs in the background.

Dirty flagging, rather than immediate cascade rewriting, handles state contradictions. When a Ledger update contradicts a tree node's Fact Table or State Claims, the conflicting node is flagged. A weekly batch process re-evaluates flagged nodes, generating updated summaries with 'patch notes' appended to the existing node rather than overwriting it. This preserves the historical record while ensuring the current map reflects current truth.

3.8 Cost, Latency, and Operational Modes

DRT-RAG is intentionally heavier than retrieval-only memory systems optimized for LongMemEval-style episodic recall. It pays for *governability*: span-anchored provenance, trust invariants, Current State Ledger arbitration, and deletion-ready semantics. These features impose overhead in both compute and engineering complexity. This paper does not present a cost benchmark. However, because the architecture is explicitly positioned for domains where correctness, auditability, and reversible memory operations matter, it is important to make the cost surface and control knobs explicit rather than leaving them implicit.

Accordingly, DRT-RAG makes two operational commitments. First, the highest-cost operations (summarization, layer formation, and most re-evaluation) are strictly asynchronous and can be frequency- and model-sized independently of the interactive experience. Second, query-time navigation cost is bounded by explicit operational modes: the system can run in a lightweight posture for everyday interaction, and only pay the full governance cost when a query or action

crosses a risk threshold (e.g., state updates, sensitive personal inferences, or any answer requiring explicit provenance).

Cost driver	When it occurs	Control knob / mitigation
L0 capture + storage growth	Every interaction	Retention policy; cold storage for older L0; encryption/compression at rest
L0→L1 summarization	Batch (scheduled or threshold-triggered)	Run off the critical path; adjust cadence; choose smaller summarizer model; cap per-run token budget
Layer growth (Ln→Ln+1)	Rare; when top-layer exceeds threshold	Tune threshold; run asynchronously; cluster on structured channels to reduce retries/rework
Librarian routing hops	Query-time (interactive)	Use a small fast routing model; bound hops by mode; cache hot trunk and top-layer map
Branch-scoped hybrid search	Query-time; within nominated branches	Limit candidate branches; cap k; prefer structured keys (Fact Table) before deep L0 scans
Re-ranking / critic pass	Query-time; optional per mode	Use smallest viable critic; enable only for high-stakes/risk-tier queries
L0 precision retrieval (verbatim quotes)	Query-time; only when attribution is required	Trigger by intent classification; retrieve only via Evidence Pointers (no global scans)
Deletion propagation + re-summarization	Rare; user-triggered; proportional to downstream references	Process off the critical path; leverage pointer graph for targeted invalidation; schedule re-summarization in batches

Operational modes provide a simple, explicit way to bound interactive cost without weakening the underlying invariants: **provenance requirements, certainty tags, and the non-upgrading trust invariants remain mandatory in all modes**; what changes by mode is only how much search depth, re-ranking, and verbatim verification the Librarian performs before answering.

- **Lite:** Ledger + top-layer map + single-branch drilldown; minimal hybrid search; no critic pass; designed for low-stakes conversational continuity and fast responses.
- **Standard:** Dual branch nomination + branch-scoped hybrid search + lightweight re-ranking; designed for typical knowledge work where both accuracy and responsiveness matter.
- **Governed:** Full invariants enforced with strict re-ranking/commitment checks; deeper drilldowns; provenance-first quoting when required; designed for state updates, commitments, safety-relevant queries, or any output that must be auditable.

The intended deployment posture is that most everyday interactions run in Lite or Standard mode, while Governed mode is reserved for moments where the cost of a wrong answer is higher than the cost of additional navigation and verification. This is not an implementation

detail; it is part of the architectural claim. DRT-RAG's governance properties are valuable precisely because they can be invoked selectively, rather than imposed uniformly on every turn.

4. Limitations and Mitigations

4.1 Information Degradation (The Telephone Game)

Recursive summarization inevitably loses nuance. Subtle emotional undertones, hedged statements, and contextual qualifications that survive at L1 may be entirely absent by L3. Biases introduced by the summarizer model at L1 compound upward and become permanent 'facts' by L3.

Mitigation: The multi-channel node schema is the primary mitigation. Structured Fact Tables and State Claims channels use extractive rather than abstractive processing, preserving precision independent of narrative quality. Evidence Pointers anchor high-level summaries to verbatim source material. Narrative Summaries are explicitly treated as orientation tools that degrade predictably, not as truth stores.

4.2 Routing Blind Spots (False Negatives)

If the top-layer Narrative Summary omits a concept — due to its relative weight at summarization time being low, even if it proves important later — the LLM router will not identify the corresponding branch as relevant, and will not retrieve the data even though it exists at L0.

Mitigation: Branch-scoped hybrid search (lexical keyword + dense embeddings) runs within all candidate branches identified by the router, catching entities and details absent from summaries. The global Hail Mary fallback handles the remaining cases. Evaluation benchmarks should explicitly test false-negative rates with queries about entities that were present in L0 but had low summarization weight. A related class of failure — rare-critical memory — deserves explicit acknowledgment: a single high-stakes event that occurs once and is never repeated may be compressed away precisely because its low recurrence gives it low summarization weight, even though it represents the most important thing in the memory. Open Loops partially address this for unresolved threads, but a resolved one-time event of high personal significance has no equivalent mechanism. Explicit user-pinned memory items — entries that are excluded from the summarization weight heuristic and preserved at all layers regardless of recurrence — are a natural extension of the architecture and a priority for future specification.

4.3 Update Cascades

New information that contradicts existing high-level summaries would, in a naive implementation, require rewriting the entire tree from the point of contradiction upward. This is computationally prohibitive and risks introducing new errors during the rewrite.

Mitigation: The Current State Ledger handles the majority of contradiction cases by maintaining current truth separately from historical record. Dirty flagging with batch re-evaluation

handles cases requiring tree updates, using an incremental patch-notes approach rather than full node rewrite.

4.4 Routing Latency

Multi-hop agentic navigation introduces latency that single-call vector retrieval does not. A query requiring three hops through the tree before answer generation may take 10-15 seconds where a flat RAG response would take 2.

Mitigation: Tiered model architecture (fast routing model, full model for answer generation) reduces latency for the navigation steps. Caching of the 'hot trunk' — the most recent 30-90 days of L1 summaries maintained in rapid-access storage — reduces retrieval time for the most common query targets. In addition, operational modes (Section 3.8) allow deployments to bound worst-case interactive latency by reserving deeper drilldown, strict re-ranking, and provenance-first quoting for high-stakes queries rather than for every turn.

4.5 Semantic Boundary Problems

Chronological chunking (daily summaries) splits semantically coherent conversations that cross arbitrary time boundaries. A decision-making process that spans three weeks of intermittent conversation may be fragmented across multiple L1 nodes, reducing the quality of any individual node's synthesis.

Mitigation: Session-aware chunking detects conversational continuity signals (topic persistence, entity recurrence) and groups L0 logs by semantic coherence rather than strict chronology where feasible. Open Loops in the node schema explicitly track unresolved threads across chunk boundaries, maintaining continuity signals even when content is chunked.

5. Privacy, Retention, and the Right to Be Forgotten

Any system designed to maintain a persistent, multi-year structured record of a person's interactions, decisions, relationships, and emotional states operates in territory with serious privacy implications. A DRT-RAG instance that has been running for two years contains a more detailed model of a person's life than most people have ever articulated consciously. This creates obligations that must be architected in, not bolted on.

5.1 Data Retention and Deletion Semantics

DRT-RAG must support meaningful deletion — not just flagging, but actual removal — across all layers simultaneously. This is architecturally non-trivial because a single L0 log entry may be referenced by Evidence Pointers in multiple L1 nodes, which are summarized into L2 nodes, whose content may have influenced Ledger entries. A deletion request for an L0 entry requires: invalidation of all Evidence Pointers referencing that entry, re-evaluation of any Fact Table items that were span-anchored to that entry (which may need to be downgraded to 'inference' or removed), flagging of affected summary nodes as 'partially redacted' with their re-summarization

scheduled, and removal or anonymization of any Ledger entries whose sole provenance was the deleted L0 source.

This is not a simple cascade delete. It is a provenance-aware redaction process. The Evidence Pointer architecture that enables audit trails also enables precise surgical deletion — the immutable pointer structure means you can trace exactly which downstream nodes were informed by any given L0 entry. This is the correct design for GDPR Article 17 compliance and equivalent regulations. Note: this deletion architecture addresses the DRT-RAG episodic tier only. Deletion propagation to any consolidated parametric layer built on top of the episodic substrate — including the LoRA adapter architecture proposed in Paper Two — requires additional mechanisms. Paper Three introduces a tiered parametric unlearning policy covering the full propagation chain from L0 deletion through adapter suppression or rollback.

5.2 Encryption and Access Control

L0 logs and all summary layers must be encrypted at rest. Access to the DRT-RAG store should be gated by user-held keys where the architecture permits — meaning the system operator cannot read a user's memory tree without the user's explicit key. The Ledger, as the fast-path authoritative store, requires particular protection: a Ledger injection attack (writing false current-state entries) is a high-impact, low-detection-risk attack surface that must be protected by both encryption and write-access controls.

5.3 Audit Logs

All write operations to the tree — new L0 entries, summarization events, Ledger updates, dirty flag operations, layer generation triggers — should be logged in a separate, append-only audit log. This audit log is the meta-memory of the memory system: it records not what was said, but what the system did with what was said. It enables debugging of summary drift, detection of anomalous write patterns, and forensic reconstruction of how a given high-level summary was produced. The audit log is itself subject to the same deletion semantics as the tree.

6. Evaluation Framework

A credible evaluation of DRT-RAG requires benchmarks designed for long-horizon interactive memory. Several such benchmarks now exist, including LongMemEval (Wu et al., ICLR 2025), which provides a standardized measure of episodic recall and longitudinal reasoning (Track 1). However, no single benchmark suite currently evaluates the full set of *governability* properties that DRT-RAG treats as first-class architectural guarantees — span-anchored provenance, certainty discipline (including the non-upgrading invariant), explicit current-state arbitration via the Ledger, and provenance-aware deletion propagation. Accordingly, we treat LongMemEval as a baseline target for retrieval-centric performance while proposing additional evaluation dimensions and adversarial task types tailored to DRT-RAG’s invariants:

Evaluation Dimension	Task Type	DRT-RAG Prediction	Baseline
Global thematic reasoning	'How has my relationship with X changed over time?'	Strong — top-layer map provides thematic orientation	Flat RAG, RAPTOR
Precision recall	'What was the exact date/name/number from conversation Y?'	Moderate-to-Strong — strongest with hybrid retrieval + Evidence Pointers	Flat RAG, agentmemory, Mastra OM
Contradiction handling	Query after a stated fact has changed	Strong — Ledger + dirty flags	RAPTOR, MemGPT, Mastra OM, Hindsight
False-negative rate	Query about entity with low summarization weight	Moderate — branch-scoped search as mitigation	Flat RAG, agentmemory, Mastra OM
Long-horizon trend	Query spanning >12 months of history	Strong — dynamic layers prevent context overflow	Flat RAG, RAPTOR, MemGPT, agentmemory, Mastra OM, Hindsight
Attribution/audit	'Which conversation led to this conclusion?'	Strong — full evidence pointer traceability	GraphRAG
Provenance-aware deletion propagation	Delete an L0 span and verify downstream invalidations and redaction markers	Strong — Evidence Pointers enable targeted invalidation + scheduled re-summarization	Flat RAG, RAPTOR, MemGPT (deletion semantics typically unspecified)

Diagnostic focus: the most revealing tests of DRT-RAG's thesis are (i) false-negative / rare-critical retrieval failures under low-salience summarization, (ii) knowledge-update correctness under conflicting history via Ledger arbitration with provenance constraints, and (iii) provenance-aware deletion propagation correctness (redaction and downstream invalidation) where required.

Prediction calibration: DRT-RAG is expected to be strongest on **knowledge updates** (where the Ledger's conflict-set semantics prevent stale summaries from overriding recent changes)

and **abstention** (where provenance and certainty invariants discourage confident claims without anchored support). We expect retrieval-only systems engineered purely for benchmark recall to outperform DRT-RAG on some single-session information extraction cases, especially in early implementations that do not include hybrid lexical+dense retrieval and re-ranking. This tradeoff is intentional: DRT-RAG's contribution is to make episodic memory governable — auditable, deletable, and safe to consolidate — not to win a single benchmark in isolation.

We additionally propose adversarial test cases explicitly designed to probe the blind-spot routing failure: queries about entities that were present in L0 but received low weight during summarization. This is the highest-risk failure mode in DRT-RAG and must be a primary focus of empirical evaluation.

7. Conclusion and Forward Reference

DRT-RAG advances the state of agentic memory through three coordinated innovations: multi-channel node schemas that resist information degradation, dynamic recursive layer generation that supports *unbounded history with bounded working context (under configured budgets)*, and an agentic LLM Librarian that navigates memory with intent rather than statistical similarity.

It is positioned most honestly not as a complete solution to AI memory, but as the episodic memory tier of a larger architecture. It excels at preserving, organizing, and retrieving the factual record of an agent's history. What it does not address is the question of how that history transforms the agent itself — how experience flows from external retrieval into something more like genuine knowledge, or how an agent develops consistent values and behavioral patterns that persist across sessions without needing to re-retrieve them.

Importantly, DRT-RAG stands alone as a governed episodic memory architecture: it can be deployed purely as a retrieval-and-governance layer without any parametric consolidation, which is an optional downstream mechanism rather than a dependency for correctness.

> *The tree preserves the facts. The next question is how facts become character.*

That question is the subject of Paper Two in this series: 'The Goal Isn't Controlled AI — It's Well-Raised AI,' which addresses the architecture of AI identity formation through sleep-state memory consolidation. Paper Three will then demonstrate how DRT-RAG and the consolidation architecture function as complementary tiers — with DRT-RAG's survival-of-summarization providing the selection signal that Paper Two identifies as the critical unsolved problem.

The complete three-paper series proposes a unified memory architecture for AI agents that operates across the full hierarchy of human memory: episodic retrieval (DRT-RAG), identity consolidation (Paper Two), and the integration of both into a system capable of genuine longitudinal reasoning, persistent values, and behavioral continuity at a level not yet demonstrated reliably in deployed AI.

Acknowledgements

Portions of this work were developed with the assistance of large language model tools including Claude (Anthropic), GPT (OpenAI), and Gemini (Google). All architectural claims, editorial judgments, and final content are the responsibility of the author.

References

Asai, A. et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique Through Self-Reflection. ICLR 2024 (Oral). arXiv:2310.11511.

Singh, A. et al. (2025). Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. arXiv:2501.09136.

Du, M. et al. (2026). A-RAG: Agentic Retrieval-Augmented Generation with Hierarchical Retrieval Interfaces. arXiv:2602.03442.

Xu, W. et al. (2025). A-MEM: Agentic Memory for LLM Agents. NeurIPS 2025. arXiv:2502.12110.

Sarthi, P. et al. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. ICLR 2024. arXiv:2401.18059.

Packer, C. et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

Edge, D. et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

Liu, N. F. et al. (2023). Lost in the Middle: How Language Models Use Long Contexts. Transactions of the Association for Computational Linguistics (TACL). arXiv:2307.03172.

Park, J. S. et al. (2023). Generative Agents: Interactive Simulacra of Human Behavior. arXiv:2304.03442.

Jimenez Gutierrez, B. et al. (2024). HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. NeurIPS 2024. arXiv:2405.14831.

Wu, D. et al. (2024). LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025. arXiv:2410.10813.

d'Anjou, A. (2026). The Goal Isn't Controlled AI — It's Well-Raised AI: Identity Formation Through Sleep-State Memory Consolidation. [Paper Two of this series].

McCann, J. (2026). agentmemory V4 — World Record on LongMemEval (96.20%, 481/500) under real-retrieval conditions (single deterministic run; USE_DIRECT_CONTEXT=False). GitHub repository: JordanMcCann/agentmemory

(FINAL_REPORT_OPUS6_WORLDRECORD.md). https://github.com/JordanMcCann/agentmemory

Barnes, T. (2026). Observational Memory: 95% on LongMemEval. Mastra Research (Feb 9, 2026; accessed Apr 2026). Reports 84.23% on LongMemEval-S with GPT-4o and 94.87% with GPT-5-mini under a stable, cacheable context window. https://mastra.ai/research/observational-memory

Latimer, C. et al. (2025). Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects. arXiv:2512.12818.

Rasmussen, P. et al. (2025). Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956.

Li, K. et al. (2026). TiMem: Temporal-Hierarchical Memory Consolidation for Long-Horizon Conversational Agents. arXiv:2601.02845.

d'Anjou, A. (2026). The Unified Memory Architecture: DRT-RAG and Sleep-State Consolidation as Complementary Tiers. [Paper Three of this series].